

TP : Query Builder (Projet : Gestion de Films)

Ce TP vise à vous guider dans la création d'un projet Laravel simple pour gérer une base de films, en mettant l'accent sur l'utilisation du **Générateur de Requêtes (Query Builder)** de Laravel. Le Query Builder fournit une interface pratique et fluide pour créer et exécuter des requêtes de base de données. Il peut être utilisé pour effectuer la plupart des opérations de base de données dans votre application et fonctionne parfaitement avec tous les systèmes de base de données pris en charge par Laravel. Le Query Builder utilise la liaison des paramètres PDO pour protéger votre application contre les attaques par injection SQL. Il n'est pas nécessaire de nettoyer ou d'assainir les chaînes transmises au constructeur de requêtes en tant que liaisons de requêtes.

Nous couvrirons toutes les méthodes décrites, adaptées au contexte d'une table films, en incluant des exemples avec get, first, value, find, pluck, chunk, arrêt de chunk avec false, lazy, lazyById/lazyByIdDesc, agrégats comme count, max, exists/doesntExist, expressions raw (selectRaw, whereRaw, etc.), jointures (inner, left, right), unions, clauses where (basiques, orWhere, whereNot, whereBetween, whereIn, etc.), ordre (orderBy, latest, oldest, inRandomOrder, reorder), groupement (groupBy, having), limite/offset (skip, take, limit, offset), insertions (insert simple/multiple, insertUsing, upsert), mises à jour (update, updateOrInsert, increment/decrement), suppressions (delete, truncate), et plus.

Le TP est structuré en étapes. Testez via routes ou Artisan.

1 : Création du Projet Laravel

Créez un projet nommé "gestion-films".

1. Exécutez :

```
composer create-project laravel/laravel 015-gestion-films
cd 015-gestion-films
```

2. Lancez le serveur :

```
php artisan serve
```

2 : Configuration de la Base de Données

Utilisez SQLite.

1. Dans .env :

```
DB_CONNECTION=sqlite
DB_DATABASE=database/database.sqlite
```

Créez le fichier : touch database/database.sqlite.

2. Testez : php artisan tinker puis DB::connection()->getPdo();.

3 : Création de la Table via Migration

Migration pour films : id, titre, realisateur, annee, genre, note, votes, timestamps.

1. Générez : `php artisan make:migration create_films_table`
2. Éditez database/migrations/..._create_films_table.php :

```
use Illuminate\Database\Migrations\Migration;
```

```

use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration {
    public function up(): void {
        Schema::create('films', function (Blueprint $table) {
            $table->id();
            $table->string('titre');
            $table->string('realisateur');
            $table->integer('annee');
            $table->string('genre');
            $table->decimal('note', 3, 1)->nullable();
            $table->integer('votes')->default(0);
            $table->timestamps();
        });
    }
    public function down(): void {
        Schema::dropIfExists('films');
    }
};

```

3. Migrez : `php artisan migrate`.

4 : Création d'un Controller et Routes

Controller pour tests.

1. Générez : `php artisan make:controller FilmController`
2. Dans FilmController.php, importez `use Illuminate\Support\Facades\DB;`
3. Routes dans routes/web.php :

```

use App\Http\Controllers\FilmController;

Route::get('/films/seed', [FilmController::class, 'seed']);
Route::get('/films/select-all', [FilmController::class, 'selectAll']);
Route::get('/films/first', [FilmController::class, 'first']);
Route::get('/films/value', [FilmController::class, 'value']);
Route::get('/films/find', [FilmController::class, 'find']);
Route::get('/films/pluck', [FilmController::class, 'pluck']);
Route::get('/films/pluck-key', [FilmController::class, 'pluckKey']);
Route::get('/films/chunk', [FilmController::class, 'chunk']);
Route::get('/films/chunk-stop', [FilmController::class, 'chunkStop']);
Route::get('/films/lazy', [FilmController::class, 'lazy']);
Route::get('/films/lazy-by-id', [FilmController::class, 'lazyById']);
Route::get('/films/aggregates', [FilmController::class, 'aggregates']);
Route::get('/films/exists', [FilmController::class, 'exists']);
Route::get('/films/select-columns', [FilmController::class, 'selectColumns']);
Route::get('/films/raw', [FilmController::class, 'raw']);
Route::get('/films/join', [FilmController::class, 'join']);
Route::get('/films/left-right-join', [FilmController::class, 'leftRightJoin']);
Route::get('/films/union', [FilmController::class, 'union']);
Route::get('/films/where', [FilmController::class, 'where']);
Route::get('/films/or-where', [FilmController::class, 'orWhere']);
Route::get('/films/where-not', [FilmController::class, 'whereNot']);
Route::get('/films/additional-wheres', [FilmController::class, 'additionalWheres']);
Route::get('/films/order-by', [FilmController::class, 'orderBy']);
Route::get('/films/latest-oldest', [FilmController::class, 'latestOldest']);
Route::get('/films/random-reorder', [FilmController::class, 'randomReorder']);
Route::get('/films/group-having', [FilmController::class, 'groupHaving']);
Route::get('/films/skip-take', [FilmController::class, 'skipTake']);
Route::get('/films/insert', [FilmController::class, 'insert']);
Route::get('/films/insert-using', [FilmController::class, 'insertUsing']);
Route::get('/films/upsert', [FilmController::class, 'upsert']);
Route::get('/films/update', [FilmController::class, 'update']);

```

```
Route::get('/films/update-or-insert', [FilmController::class, 'updateOrInsert']);
Route::get('/films/increment-decrement', [FilmController::class,
'incrementDecrement']);
Route::get('/films/delete', [FilmController::class, 'delete']);
Route::get('/films/truncate', [FilmController::class, 'truncate']);
```

5 : Implémentation des Méthodes

Seed de Données :

```
public function seed() {
    DB::table('films')->insert([
        ['titre' => 'Inception', 'realisateur' => 'Christopher Nolan', 'annee' =>
2010, 'genre' => 'Sci-Fi', 'note' => 8.8, 'votes' => 100, 'updated_at' => now()],
        ['titre' => 'The Matrix', 'realisateur' => 'Wachowski', 'annee' => 1999,
'genre' => 'Action', 'note' => 8.7, 'votes' => 50, 'updated_at' => now()-
>subDays(1)],
        ['titre' => 'Pulp Fiction', 'realisateur' => 'Quentin Tarantino', 'annee'
=> 1994, 'genre' => 'Crime', 'note' => 8.9, 'votes' => 200, 'updated_at' =>
now()->subMonth()],
        ['titre' => 'Interstellar', 'realisateur' => 'Christopher Nolan', 'annee'
=> 2014, 'genre' => 'Sci-Fi', 'note' => 8.6, 'votes' => 150, 'updated_at' =>
null],
        ['titre' => 'Dune', 'realisateur' => 'Denis Villeneuve', 'annee' => 2021,
'genre' => 'Sci-Fi', 'note' => 8.0, 'votes' => 80, 'updated_at' => now()],
    ]);
    return response()->json(['message' => 'Données insérées']);
}
```

Exécutez /films/seed une fois.

Exécution de Requêtes de Base de Données :

- **Récupération de Toutes les Lignes d'une Table :**

```
public function selectAll() {
    $films = DB::table('films')->get(); // Retourne une Collection
    foreach ($films as $film) {
        echo $film->titre;
    }
    return response()->json($films);
}
```

- **Récupération d'une Seule Ligne/Colonne d'un Tableau :**

```
public function first() {
    $film = DB::table('films')->where('titre', 'Inception')->first(); // Objet
stdClass
    return response()->json($film ? $film->realisateur : null);
}
public function value() {
    $note = DB::table('films')->where('titre', 'Inception')->value('note');
    return response()->json($note);
}
public function find() {
    $film = DB::table('films')->find(1); // Par ID
    return response()->json($film);
}
```

- **Récupération d'une Liste de Valeurs de Colonnes :**

```
public function pluck() {
```

```

    $titres = DB::table('films')->pluck('titre');
    foreach ($titres as $titre) {
        echo $titre;
    }
    return response()->json($titres);
}
public function pluckKey() {
    $titres = DB::table('films')->pluck('titre', 'realisateur');
    foreach ($titres as $realisateur => $titre) {
        echo $titre;
    }
    return response()->json($titres);
}
}

```

- **Découpage des Résultats :**

```

public function chunk() {
    DB::table('films')->orderBy('id')->chunk(100, function ($films) {
        foreach ($films as $film) {
            // Traitement
        }
    });
    return response()->json(['message' => 'Chunk exécuté']);
}
public function chunkStop() {
    DB::table('films')->orderBy('id')->chunk(100, function ($films) {
        // Traitement
        return false; // Arrête les chunks suivants
    });
    return response()->json(['message' => 'Chunk arrêté']);
}
}

```

- **Résultats en Streaming Fainéant (Lazy Streaming) :**

```

public function lazy() {
    DB::table('films')->orderBy('id')->lazy()->each(function ($film) {
        // Traitement
    });
    return response()->json(['message' => 'Lazy exécuté']);
}
public function lazyById() {
    DB::table('films')->where('votes', '<', 100)
        ->lazyById()->each(function ($film) {
            DB::table('films')->where('id', $film->id)->update(['votes' => 100]);
        });
    return response()->json(['message' => 'LazyById exécuté']);
}
}

```

Les Agrégats :

```

public function aggregates() {
    $count = DB::table('films')->count();
    $maxNote = DB::table('films')->max('note');
    $avgNote = DB::table('films')->avg('note');
    $sumVotes = DB::table('films')->sum('votes');
    $minAnnee = DB::table('films')->min('annee');
    return response()->json(['count' => $count, 'max' => $maxNote, 'avg' =>
    $avgNote, 'sum' => $sumVotes, 'min' => $minAnnee]);
}

```

Déterminer l'Existence des Enregistrements :

```

public function exists() {
    $hasHighNote = DB::table('films')->where('note', '>', 8.5)->exists();
}

```

```

    $noLowNote = DB::table('films')->where('note', '<', 5.0)->doesntExist();
    return response()->json(['hasHighNote' => $hasHighNote, 'noLowNote' =>
    $noLowNote]);
}

```

Les Instructions SELECT :

- **Spécifier une Clause SELECT :**

```

public function selectColumns() {
    $films = DB::table('films')->select('titre', 'realisateur as
    realisateur_film')->get();
    return response()->json($films);
}

```

Expressions Brutes :

```

public function raw() {
    $counts = DB::table('films')->select(DB::raw('count(*) as film_count,
    genre'))->groupBy('genre')->get();
    return response()->json($counts);
}

```

Les Méthodes Brutes : Incluez selectRaw, whereRaw, orWhereRaw, havingRaw, orHavingRaw, orderByRaw, groupByRaw dans des méthodes dédiées si needed.

Joins :

- **Clause de Jonction Interne :**

```

public function join() {
    $result = DB::table('films')
        ->join('acteurs', 'films.id', '=', 'acteurs.film_id')
        ->select('films.*', 'acteurs.nom')
        ->get();
    return response()->json($result);
}

```

- **Left Join / Right Join :**

```

public function leftRightJoin() {
    $left = DB::table('films')->leftJoin('acteurs', 'films.id', '=',
    'acteurs.film_id')->get();
    $right = DB::table('films')->rightJoin('acteurs', 'films.id', '=',
    'acteurs.film_id')->get();
    return response()->json(['left' => $left, 'right' => $right]);
}

```

Unions :

```

public function union() {
    $first = DB::table('films')->whereNull('updated_at');
    $films = DB::table('films')->where('annee', '>', 2010)->union($first)->get();
    return response()->json($films);
}

```

Clauses Where de Base :

- **La Clause Where :**

```
public function where() {
  $films = DB::table('films')->where('votes', '=', 100)->where('annee', '>', 2000)->get();
  // Ou avec tableau : ->where(['votes', '=', 100], ['annee', '>', 2000])
  return response()->json($films);
}
```

- **La Clause orWhere :**

```
public function orWhere() {
  $films = DB::table('films')->where('votes', '>', 100)->orWhere('note', '>', 8.5)->get();
  return response()->json($films);
}
```

- **La Clause WhereNot :**

```
public function whereNot() {
  $films = DB::table('films')->whereNot(function ($query) {
    $query->where('genre', 'Sci-Fi')->orWhere('note', '<', 8.0);
  })->get();
  return response()->json($films);
}
```

- **Des Clauses Where Additionnelles :**

```
public function additionalWheres() {
  $films = DB::table('films')
    ->whereBetween('votes', [50, 150])
    ->whereNotBetween('annee', [1990, 2000])
    ->whereIn('genre', ['Sci-Fi', 'Action'])
    ->whereNotIn('genre', ['Crime'])
    ->whereNull('updated_at')
    ->whereNotNull('created_at')
    ->whereDate('created_at', now()->format('Y-m-d'))
    ->whereDay('created_at', now()->day)
    // Ajoutez whereMonth, whereYear si pertinent
    ->get();
  return response()->json($films);
}
```

Ordre, Regroupement, Limite et Offset :

- **OrderBy :**

```
public function orderBy() {
  $films = DB::table('films')->orderBy('annee', 'desc')->get();
  return response()->json($films);
}
```

- **Latest et Oldest :**

```
public function latestOldest() {
  $latest = DB::table('films')->latest('updated_at')->first();
  $oldest = DB::table('films')->oldest('created_at')->first();
  return response()->json(['latest' => $latest, 'oldest' => $oldest]);
}
```

- **inRandomOrder :**

```
public function randomReorder() {
```

```

    $random = DB::table('films')->inRandomOrder()->first();
    $query = DB::table('films')->orderBy('note');
    $reordered = $query->reorder()->get();
    return response()->json(['random' => $random, 'reordered' => $reordered]);
}

```

- **GroupBy et Having :**

```

public function groupHaving() {
    $groups = DB::table('films')->groupBy('genre')->having('votes', '>', 100)->get();
    return response()->json($groups);
}

```

- **Skip et Take :**

```

public function skipTake() {
    $films = DB::table('films')->skip(2)->take(3)->get(); // Équivalent à
    offset(2)->limit(3)
    return response()->json($films);
}

```

INSERT :

```

public function insert() {
    DB::table('films')->insert(['titre' => 'New Film', 'realisateur' =>
    'Director', 'annee' => 2025, 'genre' => 'Drama', 'note' => 7.5]);
    DB::table('films')->insert([
        ['titre' => 'Film A', 'realisateur' => 'Dir A', 'annee' => 2020, 'genre'
    => 'Action', 'note' => 8.0],
        ['titre' => 'Film B', 'realisateur' => 'Dir B', 'annee' => 2021, 'genre'
    => 'Comedy', 'note' => 7.0],
    ]);
    return response()->json(['message' => 'Insérés']);
}
public function insertUsing() {
    DB::table('films_archive')->insertUsing(
        ['titre', 'annee', 'note'],
        DB::table('films')->select('titre', 'annee', 'note')->where('annee', '<',
    2000)
    );
    return response()->json(['message' => 'InsertUsing effectué']);
}

```

Upsert :

```

public function upsert() {
    DB::table('films')->upsert(
        [['titre' => 'Inception', 'note' => 9.2], ['titre' => 'New', 'note' =>
    7.0]],
        ['titre'],
        ['note']
    );
    return response()->json(['message' => 'Upsert effectué']);
}

```

UPDATE :

```

public function update() {
    $affected = DB::table('films')->where('id', 1)->update(['note' => 9.0]);
    return response()->json(['affected' => $affected]);
}
public function updateOrInsert() {

```

```

        DB::table('films')->updateOrCreate(
            ['titre' => 'Inception'],
            ['note' => 9.2]
        );
        return response()->json(['message' => 'UpdateOrCreate effectué']);
    }
    public function incrementDecrement() {
        DB::table('films')->increment('votes', 5);
        DB::table('films')->decrement('votes', 1, ['note' => 8.0]);
        return response()->json(['message' => 'Ajusté']);
    }
}

```

DELETE :

```

public function delete() {
    $deleted = DB::table('films')->where('votes', '>', 100)->delete();
    return response()->json(['deleted' => $deleted]);
}
public function truncate() {
    DB::table('films')->truncate();
    return response()->json(['message' => 'Tronquée']);
}

```

Autres (Statement, Transactions, Subquery) : Restent inchangés de la version précédente pour complétude.

6 : Ajouts et Best Practices

- **Debug :** Utilisez `->toSql()` pour voir la SQL générée.
- **Sécurité :** Liaisons PDO automatiques.
- **Pour Gros Datasets :** Préférez `chunk/lazy`.
- **Extension :** Passez à Eloquent pour modèles complexes.